

Support Vector Machines Notes

Author

K. Yu

2026

Contents

1	Kernel Methods	3
1.1	Feature Maps	3
1.2	Least-Mean Squares with Features	4
1.3	Why the Feature-Space LMS Update can be expensive	5
1.4	The Kernel Trick	5
1.5	Representing $\mathbf{w}$ as a linear combination of training features	5
1.6	Updating the coefficients instead of $\mathbf{w}$	6
1.7	Kernel Matrix Form	6
1.8	Prediction using kernels	7
1.9	Polynomial kernel example	7
1.10	Properties of kernels	8
1.11	Examples of kernels	8
1.12	Kernels as similarity measures	9
1.13	Necessary conditions for valid kernels	10
2	Support Vector Machines	11
2.1	Motivation	11
2.2	Linear classifier notation	11
2.3	Functional margin	12
2.4	Problem with functional margin	13
2.5	Geometric margin	13
2.6	Optimal margin classifier	15
2.7	Lagrange duality	16
2.8	Strong duality and KKT conditions	18
2.9	Hard-margin SVM dual	18
2.10	Support Vectors	20
2.11	Recovering b	21
2.12	Prediction using the dual form	21
2.13	Kernelised SVM	22
2.14	Regularisation and the non-separable case	22
2.15	KKT conditions for soft-margin SVM	24
2.16	Coordinate ascent	24
2.17	SMO algorithm	25

2.18 Two-variable SMO update	25
3 References and Further Reading	27

1 Kernel Methods

In ordinary linear regression, we model the output y as a linear function of the input attribute x . For example, if x denotes the living area of a house and y denotes the house price, a simple model is

$$h_{w,b}(x) = wx + b.$$

However, the relationship between x and y may be nonlinear. For example, a cubic function may fit the data better:

$$h(x) = w_3x^3 + w_2x^2 + w_1x + b.$$

Although this function is nonlinear in x , it is still linear in a transformed feature representation of x .

1.1 Feature Maps

Definition 1.1.1 (Feature map). Let $\mathcal{X}$ be the input attribute space. A **feature map** is a function $\phi : \mathcal{X} \rightarrow \mathbb{R}^p$ that maps an original input attribute $\mathbf{x} \in \mathcal{X}$ into a feature vector $\phi(\mathbf{x}) \in \mathbb{R}^p$. The original values $\mathbf{x}$ are called **input attributes**, while the transformed quantities $\phi(\mathbf{x})$ are called **feature variables**.

Example 1.1.1 (Cubic feature map). For a scalar input $x \in \mathbb{R}$, define

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \in \mathbb{R}^4.$$

Let $\mathbf{w} = [b \quad w_1 \quad w_2 \quad w_3]^\top$. Then

$$\mathbf{w}^\top \phi(x) = b + w_1x + w_2x^2 + w_3x^3.$$

Therefore, a cubic function of x can be viewed as a linear function in the transformed feature space.

Key idea

Instead of fitting a nonlinear function directly in the original input space, we map the input into a higher-dimensional feature space and fit a linear model there:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}).$$

If a constant feature 1 is included in $\phi(\mathbf{x})$, then the bias b is absorbed into $\mathbf{w}$. Otherwise, we can write

$$h_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b.$$

1.2 Least-Mean Squares with Features

We now derive least mean squares (LMS) learning when the model uses transformed features. Suppose the training set is $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$, $y \in \mathbb{R}$. Let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$ be a feature map. We fit the model

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}), \quad \mathbf{w} \in \mathbb{R}^p.$$

The least squares objective is

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right)^2. \quad (1)$$

Lemma 1.2.1. *The gradient of J is*

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = - \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \phi(\mathbf{x}^{(i)}).$$

Proof. For one training example, define

$$J^{(i)}(\mathbf{w}) = \frac{1}{2} \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right)^2.$$

By chain rule,

$$\nabla_{\mathbf{w}} J^{(i)}(\mathbf{w}) = - \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \nabla_{\mathbf{w}} \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) = - \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \phi(\mathbf{x}^{(i)}).$$

Summing over all i gives:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = - \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \phi(\mathbf{x}^{(i)}).$$

Batch gradient descent update

Gradient descent uses

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} J(\mathbf{w}).$$

Substituting the gradient gives

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \phi(\mathbf{x}^{(i)}). \quad (2)$$

This is the LMS batch update in feature space.

Stochastic gradient descent update

For one training example $(\mathbf{x}^{(i)}, y^{(i)})$, the SGD update is

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \phi(\mathbf{x}^{(i)}). \quad (3)$$

1.3 Why the Feature-Space LMS Update can be expensive

The update above requires computing and storing $\phi(\mathbf{x})$. This can be very expensive if $\phi(\mathbf{x})$ is high-dimensional. For example, suppose $\mathbf{x} = [x_1 \ x_2 \ \cdots \ x_d]^\top \in \mathbb{R}^d$. A degree- ≤ 3 polynomial feature map may contain all monomials of degree 0, 1, 2, 3:

$$\phi(\mathbf{x}) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_1^2 \\ x_1x_2 \\ x_1x_3 \\ \vdots \\ x_1^3 \\ x_1^2x_2 \\ \vdots \end{bmatrix}.$$

If repeated monomials are included, then the number of features is $1 + d + d^2 + d^3$, hence $p = O(d^3)$. For example, when $d = 1000$, $d^3 = 10^9$, which is too large to explicitly store and update.

1.4 The Kernel Trick

The key observation is that many learning algorithms only depend on inner products of feature vectors $\phi(\mathbf{x})^\top \phi(\mathbf{z})$. If we can compute this inner product without explicitly computing $\phi(\mathbf{x})$, then we can work implicitly in a high-dimensional feature space.

Definition 1.4.1 (Kernel function). A function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is called a **kernel function** corresponding to a feature map ϕ if

$$K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$$

for all $\mathbf{x}, \mathbf{z} \in \mathcal{X}$.

1.5 Representing $\mathbf{w}$ as a linear combination of training features

Lemma 1.5.1. *If $\mathbf{w}$ is initialised as $\mathbf{0}$, then after every batch gradient descent update,*

$$\mathbf{w} \in \text{span}\{\phi(\mathbf{x}^{(1)}), \dots, \phi(\mathbf{x}^{(n)})\}.$$

Equivalently, there exist coefficients $a_1, \dots, a_n \in \mathbb{R}$ such that

$$\mathbf{w} = \sum_{i=1}^n a_i \phi(\mathbf{x}^{(i)}).$$

Proof (by induction). At initialisation, $\mathbf{w} = \mathbf{0}$. This can be written as

$$\mathbf{w} = \sum_{i=1}^n 0 \cdot \phi(\mathbf{x}^{(i)}).$$

So the claim holds initially. Now suppose at some iteration,

$$\mathbf{w} = \sum_{i=1}^n a_i \phi(\mathbf{x}^{(i)}).$$

One batch update,

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \phi(\mathbf{x}^{(i)}).$$

Substitute the induction hypothesis, the update can be written as

$$\begin{aligned} \mathbf{w} &\leftarrow \sum_{i=1}^n a_i \phi(\mathbf{x}^{(i)}) + \alpha \sum_{i=1}^n \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \phi(\mathbf{x}^{(i)}) \\ &= \sum_{i=1}^n \left[a_i + \alpha \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right) \right] \phi(\mathbf{x}^{(i)}). \end{aligned}$$

Therefore, the updated $\mathbf{w}$ is still a linear combination of the training feature vectors. By induction, the result holds after every update.

$$\mathbf{w} \in \text{span}\{\phi(\mathbf{x}^{(1)}), \dots, \phi(\mathbf{x}^{(n)})\}.$$

1.6 Updating the coefficients instead of $\mathbf{w}$

Because

$$\mathbf{w} = \sum_{j=1}^n a_j \phi(\mathbf{x}^{(j)}),$$

we can represent $\mathbf{w}$ indirectly using the coefficient vector $\mathbf{a} = [a_1 \ \dots \ a_n]^\top \in \mathbb{R}^n$. From the update in Lemma 1.5.1, the coefficient update is

$$a_i \leftarrow a_i + \alpha \left(y^{(i)} - \mathbf{w}^\top \phi(\mathbf{x}^{(i)}) \right).$$

Now compute

$$\mathbf{w}^\top \phi(\mathbf{x}^{(i)}) = \left(\sum_{j=1}^n a_j \phi(\mathbf{x}^{(j)}) \right)^\top \phi(\mathbf{x}^{(i)}) = \sum_{j=1}^n a_j \phi(\mathbf{x}^{(j)})^\top \phi(\mathbf{x}^{(i)}).$$

Using the kernel function $K(\mathbf{x}^{(j)}, \mathbf{x}^{(i)}) = \phi(\mathbf{x}^{(j)})^\top \phi(\mathbf{x}^{(i)})$, we write the update as

$$a_i \leftarrow a_i + \alpha \left(y^{(i)} - \sum_{j=1}^n a_j K(\mathbf{x}^{(j)}, \mathbf{x}^{(i)}) \right). \quad (4)$$

1.7 Kernel Matrix Form

Definition 1.7.1 (Kernel matrix). Given training inputs $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}$, the **kernel matrix** is the matrix $K \in \mathbb{R}^{n \times n}$ defined by

$$[K]_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

for all $i, j \in \{1, \dots, n\}$.

Batch update in vector form

Let $\mathbf{y} = [y^{(1)} \ \cdots \ y^{(n)}]^\top$, $\mathbf{a} = [a_1 \ \cdots \ a_n]^\top$. Then the prediction vector on the training set is

$$K\mathbf{a} = \begin{bmatrix} K(\mathbf{x}^{(1)}, \mathbf{x}^{(1)}) & \cdots & K(\mathbf{x}^{(1)}, \mathbf{x}^{(n)}) \\ \vdots & \ddots & \vdots \\ K(\mathbf{x}^{(n)}, \mathbf{x}^{(1)}) & \cdots & K(\mathbf{x}^{(n)}, \mathbf{x}^{(n)}) \end{bmatrix} \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n a_i K(\mathbf{x}^{(1)}, \mathbf{x}^{(i)}) \\ \vdots \\ \sum_{i=1}^n a_i K(\mathbf{x}^{(n)}, \mathbf{x}^{(i)}) \end{bmatrix}.$$

Thus, the coefficient update is

$$\mathbf{a} \leftarrow \mathbf{a} + \alpha(\mathbf{y} - K\mathbf{a}). \quad (5)$$

This is the kernelised LMS update.

1.8 Prediction using kernels

After learning the coefficients $\mathbf{a}$, we predict on a new input $\mathbf{x}$ as follows:

$$h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}).$$

Using Lemma 1.5.1, we have

$$h(\mathbf{x}) = \left(\sum_{i=1}^n a_i \phi(\mathbf{x}^{(i)}) \right)^\top \phi(\mathbf{x}).$$

Therefore,

$$h(\mathbf{x}) = \sum_{i=1}^n a_i \phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}) = \sum_{i=1}^n a_i K(\mathbf{x}^{(i)}, \mathbf{x}).$$

If we explicitly include a bias term, then

$$h(\mathbf{x}) = \sum_{i=1}^n a_i K(\mathbf{x}^{(i)}, \mathbf{x}) + b.$$

1.9 Polynomial kernel example

Consider the degree-3 polynomial-style feature map containing all monomials of degree at most 3, including repetitions. Then

$$\phi(\mathbf{x})^\top \phi(\mathbf{z}) = 1 + \underbrace{\sum_{i=1}^d x_i z_i}_{=\mathbf{x}^\top \mathbf{z}} + \underbrace{\sum_{i=1}^d \sum_{j=1}^d x_i x_j z_i z_j}_{=(\mathbf{x}^\top \mathbf{z})^2} + \underbrace{\sum_{i=1}^d \sum_{j=1}^d \sum_{k=1}^d x_i x_j x_k z_i z_j z_k}_{=(\mathbf{x}^\top \mathbf{z})^3}.$$

Therefore

$$K(\mathbf{x}, \mathbf{z}) = 1 + \mathbf{x}^\top \mathbf{z} + (\mathbf{x}^\top \mathbf{z})^2 + (\mathbf{x}^\top \mathbf{z})^3.$$

This kernel can be computed in $O(d)$ time, even though the corresponding feature space has dimension $O(d^3)$.

1.10 Properties of kernels

The kernel function contains all the information needed about the feature map. Once we know $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$, the algorithm no longer needs to explicitly compute $\phi(\mathbf{x})$. This leads to an important question:

What functions $K(\mathbf{x}, \mathbf{z})$ are valid kernel functions?

That is, when does there exist a feature map ϕ such that $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$?

1.11 Examples of kernels

Example 1.11.1 (Homogeneous quadratic kernel). Let $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2$. Then

$$K(\mathbf{x}, \mathbf{z}) = \left(\sum_{i=1}^d x_i z_i \right) \left(\sum_{j=1}^d x_j z_j \right).$$

Expanding,

$$K(\mathbf{x}, \mathbf{z}) = \sum_{i=1}^d \sum_{j=1}^d x_i x_j z_i z_j.$$

This can be rewritten as

$$K(\mathbf{x}, \mathbf{z}) = \sum_{i,j} (x_i x_j) (z_i z_j).$$

Therefore, K corresponds to a feature map containing all degree-2 monomials

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ \vdots \\ x_i x_j \\ \vdots \\ x_d x_d \end{bmatrix}.$$

For $d = 3$, this feature map is

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \end{bmatrix}.$$

Although $\phi(\mathbf{x})$ has dimension d^2 , the kernel can be computed in $O(d)$ time by first computing $\mathbf{x}^\top \mathbf{z}$.

Example 1.11.2 (Inhomogeneous quadratic kernel). Consider $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^2$. Expanding,

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2 + 2c\mathbf{x}^\top \mathbf{z} + c^2.$$

Therefore,

$$K(\mathbf{x}, \mathbf{z}) = \sum_{i=1}^d \sum_{j=1}^d (x_i x_j)(z_i z_j) + \sum_{i=1}^d (x_i \sqrt{2c})(z_i \sqrt{2c}) + c \cdot c.$$

So the corresponding feature map contains:

1. all second-order terms $x_i x_j$;
2. all first-order terms $x_i \sqrt{2c}$;
3. one constant feature c .

For $d = 3$,

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \\ x_1 \sqrt{2c} \\ x_2 \sqrt{2c} \\ x_3 \sqrt{2c} \\ c \end{bmatrix}.$$

The parameter c controls the relative weighting between first-order features and second-order features.

Example 1.11.3 (General polynomial kernel). More generally,

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^k$$

corresponds to a feature space containing monomials up to degree k . The dimension of this feature space grows roughly as $O(d^k)$, but the kernel can still be computed in $O(d)$ time, because we only need to compute $\mathbf{x}^\top \mathbf{z}$, add c , and raise the result to the power k .

1.12 Kernels as similarity measures

A kernel can also be interpreted as a similarity measure. If $\phi(\mathbf{x})$ and $\phi(\mathbf{z})$ point in similar directions, then their inner product is large. If they are close to orthogonal, then the inner product is small. Thus, $K(\mathbf{x}, \mathbf{z})$ measures similarity in feature space, not necessarily in the original input space. However, this intuition is not always perfect, because inner product depend on vector norms as well as angles. Still, it is a useful way to think about kernels.

Example 1.12.1 (Gaussian kernel). The Gaussian kernel is

$$K(\mathbf{x}, \mathbf{z}) = \exp \left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2} \right)$$

where $\sigma > 0$ is a bandwidth parameter. If $\mathbf{x}$ and $\mathbf{z}$ are close, then $\|\mathbf{x} - \mathbf{z}\|^2$ is small, so $K(\mathbf{x}, \mathbf{z}) \approx 1$. If $\mathbf{x}$ and $\mathbf{z}$ are far apart, then $\|\mathbf{x} - \mathbf{z}\|^2$ is large, so $K(\mathbf{x}, \mathbf{z}) \approx 0$. The Gaussian kernel corresponds to an infinite-dimensional feature map.

1.13 Necessary conditions for valid kernels

Suppose K is a valid kernel, meaning there exists a feature map ϕ such that $K(\mathbf{x}, \mathbf{z}) = \phi(\mathbf{x})^\top \phi(\mathbf{z})$. Given any finite set of points $\{\mathbf{x}^{(i)}\}_{i=1}^n$, define the kernel matrix $K \in \mathbb{R}^{n \times n}$ by $[K]_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.

Lemma 1.13.1. *If K is a valid kernel, then the corresponding kernel matrix is symmetric.*

Proof. Since $[K]_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, and K is valid,

$$[K]_{ij} = \phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)}).$$

Using symmetry of the inner product,

$$\phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)}) = \phi(\mathbf{x}^{(j)})^\top \phi(\mathbf{x}^{(i)}).$$

Therefore,

$$[K]_{ij} = K(\mathbf{x}^{(j)}, \mathbf{x}^{(i)}) = [K]_{ji}.$$

Hence, $K = K^\top$.

Lemma 1.13.2. *If K is a valid kernel, then for any finite set of points, the kernel matrix K is positive semidefinite:*

$$\mathbf{z}^\top K \mathbf{z} \geq 0$$

for all $\mathbf{z} \in \mathbb{R}^n$.

Proof. Let $\mathbf{z} = [z_1 \ \cdots \ z_n]^\top \in \mathbb{R}^n$. Then

$$\mathbf{z}^\top K \mathbf{z} = \sum_{i=1}^n \sum_{j=1}^n z_i [K]_{ij} z_j = \sum_{i=1}^n \sum_{j=1}^n z_i \phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)}) z_j$$

since K is valid. Write feature map coordinate-wise as

$$\phi(\mathbf{x}) = \begin{bmatrix} \phi_1(\mathbf{x}) \\ \phi_2(\mathbf{x}) \\ \vdots \end{bmatrix}.$$

Then

$$\phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)}) = \sum_k \phi_k(\mathbf{x}^{(i)}) \phi_k(\mathbf{x}^{(j)}).$$

Therefore,

$$\mathbf{z}^\top K \mathbf{z} = \sum_{i=1}^n \sum_{j=1}^n z_i \left(\sum_k \phi_k(\mathbf{x}^{(i)}) \phi_k(\mathbf{x}^{(j)}) \right) z_j = \sum_k \sum_{i=1}^n \sum_{j=1}^n z_i \phi_k(\mathbf{x}^{(i)}) \phi_k(\mathbf{x}^{(j)}) z_j.$$

For each coordinate k , define $a_i = z_i \phi_k(\mathbf{x}^{(i)})$. Then

$$\sum_k \sum_{i=1}^n \sum_{j=1}^n z_i \phi_k(\mathbf{x}^{(i)}) \phi_k(\mathbf{x}^{(j)}) z_j = \sum_k \sum_{i=1}^n \sum_{j=1}^n a_i a_j = \sum_k \left(\sum_{i=1}^n a_i \right)^2.$$

Hence,

$$\mathbf{z}^\top K \mathbf{z} = \sum_k \left(\sum_{i=1}^n z_i \phi_k(\mathbf{x}^{(i)}) \right)^2 \geq 0.$$

So K is positive semidefinite.

Theorem 1.13.1 (Mercer's theorem). *Let $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ be a function. Then K is a valid Mercer kernel if and only if, for any finite set $\{\mathbf{x}^{(i)}\}_{i=1}^n$, the corresponding kernel matrix $[K]_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ is symmetric positive definite.*

Mercer's theorem gives a way to check whether a function is a valid kernel without explicitly constructing the feature map ϕ .

2 Support Vector Machines

2.1 Motivation

Support Vector Machines (SVM) are based on the idea that a good classifier should not only classify the training examples correctly, but should also classify them with a large **margin**.

In logistic regression, we use

$$h(\mathbf{x}) = g(\mathbf{w}^\top \mathbf{x} + b),$$

where g is the sigmoid function. The prediction is 1 when $h(\mathbf{x}) \geq 0.5$, which is equivalent to $\mathbf{w}^\top \mathbf{x} + b \geq 0$. For a positive training example, if $\mathbf{w}^\top \mathbf{x} + b \gg 0$, then the model is very confident that $y = 1$. Similarly, if $\mathbf{w}^\top \mathbf{x} + b \ll 0$, then the model is very confident that $y = 0$.

SVM formalises this intuition using the idea of **margin**.

2.2 Linear classifier notation

For SVMs, we use labels $y \in \{-1, 1\}$ instead of $y \in \{0, 1\}$. A linear classifier is

$$h(\mathbf{x}) = g(\mathbf{w}^\top \mathbf{x} + b)$$

where

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0, \\ -1 & \text{if } z < 0. \end{cases}$$

Thus, the prediction rule is

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b).$$

The decision boundary is

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

This boundary is also called a **separating hyperplane**.

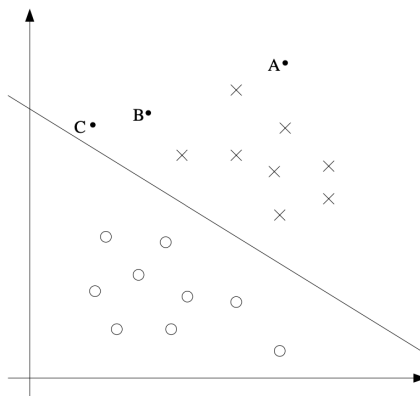


Figure 1: Positive examples, negative examples, a separating hyperplane, and points A, B, C . The point farther from the decision boundary is classified with higher confidence.

2.3 Functional margin

Definition 2.3.1 (Functional margin for one example). Given a training example $(\mathbf{x}^{(i)}, y^{(i)})$, where $y^{(i)} \in \{-1, 1\}$, the **functional margin** of $(\mathbf{w}, b)$ with respect to this example is

$$\hat{\gamma}^{(i)} = y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b).$$

If $y^{(i)} = 1$, then

$$\hat{\gamma}^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} + b.$$

For the prediction to be correct and confident, we want $\mathbf{w}^\top \mathbf{x}^{(i)} + b \gg 0$.

If $y^{(i)} = -1$, then

$$\hat{\gamma}^{(i)} = -(\mathbf{w}^\top \mathbf{x}^{(i)} + b).$$

For the prediction to be correct and confident, we want $\mathbf{w}^\top \mathbf{x}^{(i)} + b \ll 0$.

Therefore, in both cases, a large positive functional margin means the classifier is correct and confident.

Lemma 2.3.1. A training example $(\mathbf{x}^{(i)}, y^{(i)})$ is correctly classified if and only if $\hat{\gamma}^{(i)} > 0$.

Proof. The classifier predicts

$$\hat{y}^{(i)} = \text{sign}(\mathbf{w}^\top \mathbf{x}^{(i)} + b).$$

The prediction is correct if $\hat{y}^{(i)} = y^{(i)}$. This happens exactly when $y^{(i)}$ and $\mathbf{w}^\top \mathbf{x}^{(i)} + b$ have the same sign. Equivalently,

$$y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) > 0.$$

Hence, $\hat{\gamma}^{(i)} > 0$. So the example is correctly classified if and only if its functional margin is positive.

Definition 2.3.2 (Functional margin of a training set). Given a training set $S = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, the functional margin of $(\mathbf{w}, b)$ with respect to S is

$$\hat{\gamma} = \min\{\hat{\gamma}^{(i)}\}_{i=1}^n = \min\{\mathbf{w}^\top \mathbf{x}^{(i)} + b\}_{i=1}^n.$$

The training set functional margin is the smallest functional margin among all training examples.

2.4 Problem with functional margin

The functional margin is not a fully meaningful measure of confidence because it changes under scaling. For example, replace $\mathbf{w}$ by $2\mathbf{w}$, and b by $2b$. Then

$$\text{sign}(\mathbf{w}^\top \mathbf{x} + b) = \text{sign}(2\mathbf{w}^\top \mathbf{x} + 2b).$$

So the classifier does not change. However, the functional margin becomes

$$y^{(i)}(2\mathbf{w}^\top \mathbf{x}^{(i)} + 2b) = 2y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b).$$

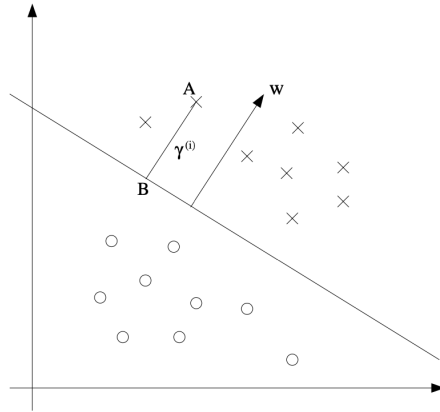
Thus, $\hat{\gamma}^{(i)}$ is scaled by 2, even though the classifier itself is unchanged. Therefore, functional margin depends on the arbitrary scale of $\mathbf{w}$ and b . This motivates the geometric margin.

2.5 Geometric margin

The geometric margin measures the actual Euclidean distance from a point to the decision boundary. The decision boundary is

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

The vector $\mathbf{w}$ is orthogonal to this hyperplane.



Definition 2.5.1 (Geometric margin for one example). For a training example $(\mathbf{x}^{(i)}, y^{(i)})$, the **geometric margin** is

$$\gamma^{(i)} = y^{(i)} \left[\left(\frac{\mathbf{w}}{\|\mathbf{w}\|} \right)^\top \mathbf{x}^{(i)} + \frac{b}{\|\mathbf{w}\|} \right].$$

Equivalently,

$$\gamma^{(i)} = \frac{y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|} = \frac{\hat{\gamma}^{(i)}}{\|\mathbf{w}\|}.$$

Lemma 2.5.1. For a corrected classified example $(\mathbf{x}^{(i)}, y^{(i)})$, the perpendicular distance from $\mathbf{x}^{(i)}$ to the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ is

$$\frac{|\mathbf{w}^\top \mathbf{x}^{(i)} + b|}{\|\mathbf{w}\|}.$$

With labels $y^{(i)} \in \{-1, 1\}$, this can be written as

$$\gamma^{(i)} = \frac{y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|}.$$

Proof. Let A be the point $\mathbf{x}^{(i)}$. Let B be the projection of A onto the decision boundary. The unit vector in the direction of $\mathbf{w}$ is

$$\frac{\mathbf{w}}{\|\mathbf{w}\|}.$$

For a positive example $y^{(i)} = 1$, the point B can be written as

$$\mathbf{x}^{(i)} - \gamma^{(i)} \frac{\mathbf{w}}{\|\mathbf{w}\|}.$$

Since B lies on the decision boundary,

$$\mathbf{w}^\top \left(\mathbf{x}^{(i)} - \gamma^{(i)} \frac{\mathbf{w}}{\|\mathbf{w}\|} \right) + b = 0.$$

Expanding,

$$\mathbf{w}^\top \mathbf{x}^{(i)} - \gamma^{(i)} \frac{\mathbf{w}^\top \mathbf{w}}{\|\mathbf{w}\|} + b = 0.$$

We get

$$\mathbf{w}^\top \mathbf{x}^{(i)} - \gamma^{(i)} \|\mathbf{w}\| + b = 0.$$

Therefore,

$$\gamma^{(i)} = \frac{\mathbf{w}^\top \mathbf{x}^{(i)} + b}{\|\mathbf{w}\|}.$$

For a negative example $y^{(i)} = -1$, the sign is corrected by multiplying by $y^{(i)}$. Thus, in general

$$\gamma^{(i)} = \frac{y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|}.$$

Definition 2.5.2 (Geometric margin of a training set). Given $S = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, the geometric margin of $(\mathbf{w}, b)$ with respect to S is

$$\gamma = \min\{\gamma^{(i)}\}_{i=1}^n = \min \left\{ \frac{y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|} \right\}_{i=1}^n.$$

Lemma 2.5.2. For any constant $c > 0$, scaling $\mathbf{w}$ and b by any scalar $c \in \mathbb{R}$ does not change the geometric margin.

Proof.

$$\gamma_c^{(i)} = \frac{y^{(i)}(c\mathbf{w}^\top \mathbf{x}^{(i)} + cb)}{\|c\mathbf{w}\|} = \frac{cy^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)}{c\|\mathbf{w}\|} = \gamma^{(i)}.$$

2.6 Optimal margin classifier

Assume the training set is linearly separable. We want to find the separating hyperplane with the largest geometric margin. The most direct formulation is

$$\begin{aligned} \max_{\gamma, \mathbf{w}, b} \quad & \gamma \\ \text{s.t.} \quad & y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq \gamma, \quad i = 1, \dots, n, \\ & \|\mathbf{w}\| = 1. \end{aligned}$$

Because $\|\mathbf{w}\| = 1$, the functional margin equals the geometric margin. However, the constraint $\|\mathbf{w}\| = 1$ is non-convex, so this formulation is not convenient.

Reformulation

Recall that

$$\gamma = \frac{\hat{\gamma}}{\|\mathbf{w}\|}.$$

So we can write the problem as

$$\max_{\hat{\gamma}, \mathbf{w}, b} \frac{\hat{\gamma}}{\|\mathbf{w}\|} \quad \text{s.t.} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq \hat{\gamma}, \quad i = 1, \dots, n,$$

Because we can freely rescale $\mathbf{w}$ and b , impose the scaling constraint $\hat{\gamma} = 1$. Then maximising

$$\frac{\hat{\gamma}}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

is equivalent to minimising $\|\mathbf{w}\|^2$. For mathematical convenience, use

$$\frac{1}{2}\|\mathbf{w}\|^2.$$

Theorem 2.6.1 (Primal hard-margin SVM). For linearly separable data, the optimal margin classifier is given by the solution to

$$\min_{\mathbf{w}, b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1, \quad i = 1, \dots, n.$$

This is the hard-margin SVM primal problem.

Proof sketch. We want to maximise the geometric margin

$$\gamma = \frac{\hat{\gamma}}{\|\mathbf{w}\|}.$$

Because the classifier does not change under positive scaling of $(\mathbf{w}, b)$, we may choose the scale so that $\hat{\gamma} = 1$. Then

$$\gamma = \frac{1}{\|\mathbf{w}\|}.$$

Maximising γ is equivalent to minimising $\|\mathbf{w}\|$, which is equivalent to minimising

$$\frac{1}{2}\|\mathbf{w}\|^2.$$

The constraint that every example has functional margin at least 1 is

$$y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1.$$

Therefore, the optimal margin classifier solves the stated quadratic program.

2.7 Lagrange duality

SVMs are usually solved through their dual form. The dual form also allows kernels to be used naturally.

Equality-constrained case

Consider

$$\min_{\mathbf{w}} f(\mathbf{w}) \quad \text{s.t.} \quad h_i(\mathbf{w}) = 0, \quad i = 1, \dots, l.$$

The Lagrangian is

$$\mathcal{L}(\mathbf{w}, \beta) = f(\mathbf{w}) + \sum_{i=1}^l \beta_i h_i(\mathbf{w}),$$

where β_i are Lagrange multipliers. A solution can be found by setting

$$\frac{\partial \mathcal{L}}{\partial w_j} = 0, \quad \frac{\partial \mathcal{L}}{\partial \beta_i} = 0.$$

Inequality and equality constraints

Consider the primal problem

$$\begin{aligned} \max_{\mathbf{w}} f(\mathbf{w}) \\ \text{s.t. } g_i(\mathbf{w}) \leq 0, \quad i = 1, \dots, k \\ h_i(\mathbf{w}) = 0, \quad i = 1, \dots, l. \end{aligned}$$

The generalised Lagrangian is

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = f(\mathbf{w}) + \sum_{i=1}^k \alpha_i g_i(\mathbf{w}) + \sum_{i=1}^l \beta_i h_i(\mathbf{w}).$$

Here, $\alpha_i \geq 0$.

Definition 2.7.1 (Primal objective through the Lagrangian). Define

$$\theta_P(\mathbf{w}) = \max_{\boldsymbol{\alpha}, \boldsymbol{\beta}: \alpha_i \geq 0} \mathcal{L}(\mathbf{w}, \boldsymbol{\alpha}, \boldsymbol{\beta}).$$

If $\mathbf{w}$ violates any constraint, then $\theta_P(\mathbf{w}) = \infty$. If $\mathbf{w}$ satisfies all constraints, then $\theta_P(\mathbf{w}) = f(\mathbf{w})$. Therefore,

$$\min_{\mathbf{w}} \theta_P(\mathbf{w})$$

is equivalent to the original primal problem.

The primal optimal value is

$$p^* = \min_{\mathbf{w}} \theta_P(\mathbf{w}).$$

Definition 2.7.2 (Dual objective). Define

$$\theta_D(\boldsymbol{\alpha}, \boldsymbol{\beta}) = \min_{\mathbf{w}} \mathcal{L}(\mathbf{w}, \boldsymbol{\alpha}, \boldsymbol{\beta}).$$

The dual problem is

$$\max_{\boldsymbol{\alpha}, \boldsymbol{\beta}: \alpha_i \geq 0} \theta_D(\boldsymbol{\alpha}, \boldsymbol{\beta}) = \max_{\boldsymbol{\alpha}, \boldsymbol{\beta}: \alpha_i \geq 0} \min_{\mathbf{w}} \mathcal{L}(\mathbf{w}, \boldsymbol{\alpha}, \boldsymbol{\beta}).$$

The dual optimal value is

$$d^* = \max_{\boldsymbol{\alpha}, \boldsymbol{\beta}: \alpha_i \geq 0} \theta_D(\boldsymbol{\alpha}, \boldsymbol{\beta}).$$

Lemma 2.7.1 (Weak duality). *For any constrained optimisation problem, $d^* \leq p^*$.*

Proof. For any function, we have

$$\max \min \leq \min \max.$$

Therefore,

$$\begin{aligned} d^* &= \max_{\boldsymbol{\alpha}, \boldsymbol{\beta}: \alpha_i \geq 0} \theta_D(\boldsymbol{\alpha}, \boldsymbol{\beta}) \\ &= \max_{\boldsymbol{\alpha}, \boldsymbol{\beta}: \alpha_i \geq 0} \min_{\mathbf{w}} \mathcal{L}(\mathbf{w}, \boldsymbol{\alpha}, \boldsymbol{\beta}) \leq \min_{\mathbf{w}} \max_{\boldsymbol{\alpha}, \boldsymbol{\beta}: \alpha_i \geq 0} \mathcal{L}(\mathbf{w}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = p^*. \end{aligned}$$

This is called **weak duality**.

2.8 Strong duality and KKT conditions

Under certain conditions, the primal and dual problems have the same optimal value. If

1. f is convex;
2. each g_i is convex;
3. each h_i is affine;
4. the inequality constraints are strictly feasible,

then $d^* = p^*$. This is strong duality.

Theorem 2.8.1 (KKT conditions). *Under the above assumptions, there exist optimal variables $\mathbf{w}^*, \boldsymbol{\alpha}^*, \beta^*$ such that the following Karush-Kuhn-Tucker conditions hold:*

1. **Stationarity:** $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^*, \boldsymbol{\alpha}^*, \beta^*) = \mathbf{0}$.
2. **Equality constraint satisfaction:** $h_i(\mathbf{w}^*) = 0, i = 1, \dots, l$.
3. **Complementary slackness:** $\alpha_i^* g_i(\mathbf{w}^*) = 0, i = 1, \dots, k$.
4. **Primal feasibility:** $g_i(\mathbf{w}^*) \leq 0, i = 1, \dots, k$.
5. **Dual feasibility:** $\alpha_i^* \geq 0, i = 1, \dots, k$.

Key interpretation: complementary slackness

The condition $\alpha_i^* g_i(\mathbf{w}^*) = 0$ means that if $\alpha_i^* > 0$, then $g_i(\mathbf{w}^*) = 0$; if $g_i(\mathbf{w}^*) < 0$, then $\alpha_i^* = 0$. In SVMs, this implies that only training examples exactly on the margin have nonzero α_i . These examples are called **support vectors**.

2.9 Hard-margin SVM dual

Recall the hard-margin primal problem

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1, \quad i = 1, \dots, n.$$

Rewrite the constraint as

$$g_i(\mathbf{w}, b) = 1 - y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \leq 0.$$

Lagrangian

The Lagrangian is

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \alpha_i \left[1 - y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \right].$$

Equivalently,

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i \left[y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) - 1 \right].$$

where $\alpha_i \geq 0$.

Lemma 2.9.1 (Stationarity with respect to $\mathbf{w}$). *At the optimum,*

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)}.$$

Proof. Differentiate the Lagrangian with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{w}^{(i)}.$$

Set this equal to zero:

$$\mathbf{w} - \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{w}^{(i)} = \mathbf{0}.$$

Therefore,

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)}.$$

Lemma 2.9.2 (Stationarity with respect to b). *At the optimum,*

$$\sum_{i=1}^n \alpha_i y^{(i)} = 0.$$

Proof. Differentiate the Lagrangian with respect to b :

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i y^{(i)}.$$

Set this equal to zero, we get

$$\sum_{i=1}^n \alpha_i y^{(i)} = 0.$$

Theorem 2.9.1 (Hard-margin SVM dual problem). *The dual of the hard-margin SVM is*

$$\begin{aligned} \max_{\boldsymbol{\alpha}} W(\boldsymbol{\alpha}) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y^{(i)} y^{(j)} \alpha_i \alpha_j (\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)} \\ \text{s.t. } \alpha_i &\geq 0, \quad i = 1, \dots, n, \\ \sum_{i=1}^n \alpha_i y^{(i)} &= 0. \end{aligned}$$

Proof. Starting from

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i \left[y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) - 1 \right],$$

expand:

$$\mathcal{L} = \frac{1}{2} \mathbf{w}^\top \mathbf{w} - \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)} - \sum_{i=1}^n \alpha_i y^{(i)} b + \sum_{i=1}^n \alpha_i.$$

Using Lemma 2.9.1, we have

$$\mathbf{w}^\top \mathbf{w} = \left(\sum_{i=1}^n \alpha_i y^{(i)} \mathbf{w}^{(i)} \right)^\top \left(\sum_{i=1}^n \alpha_i y^{(i)} \mathbf{w}^{(i)} \right).$$

Therefore,

$$\mathbf{w}^\top \mathbf{w} = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} (\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)}.$$

Also,

$$\sum_{i=1}^n \alpha_i y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)} = \mathbf{w}^\top \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)} = \mathbf{w}^\top \mathbf{w}.$$

Finally, by Lemma 2.9.2, the b -term vanishes:

$$b \sum_{i=1}^n \alpha_i y^{(i)} = 0.$$

Thus,

$$\mathcal{L} = \frac{1}{2} \mathbf{w}^\top \mathbf{w} - \mathbf{w}^\top \mathbf{w} + \sum_{i=1}^n \alpha_i.$$

Therefore,

$$\mathcal{L} = \sum_{i=1}^n \alpha_i - \frac{1}{2} \mathbf{w}^\top \mathbf{w}.$$

Substituting the expression for $\mathbf{w}^\top \mathbf{w}$, we obtain

$$W(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y^{(i)} y^{(j)} \alpha_i \alpha_j (\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)}.$$

This gives the dual problem.

2.10 Support Vectors

From KKT complementary slackness,

$$\alpha_i \left[1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \right] = 0.$$

Thus, if $\alpha_i > 0$, then $1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 0$. So

$$y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 1.$$

These are exactly the points lying on the margin boundary.

Definition 2.10.1 (Support vectors). Training examples with $\alpha_i > 0$ are called **support vectors**. They are the examples that lie on the margin and determine the separating hyperplane.

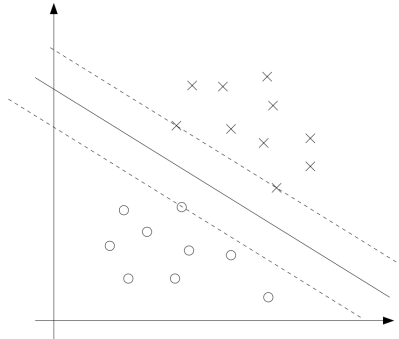


Figure 2: Support vectors are the points lying on the dashed margin lines parallel to the separating hyperplane.

2.11 Recovering b

After solving the dual and obtaining α , we can compute

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)}.$$

For the hard-margin case,

$$b^* = -\frac{1}{2} \left[\max \{ (\mathbf{w}^*)^\top \mathbf{x}^{(i)} : \forall i = 1, \dots, n, y^{(i)} = -1 \} + \min \{ (\mathbf{w}^*)^\top \mathbf{x}^{(i)} : \forall i = 1, \dots, n, y^{(i)} = 1 \} \right]$$

This places the decision boundary halfway between the closest positive and negative examples.

2.12 Prediction using the dual form

Given a new input $\mathbf{x}$, the classifier computes $\mathbf{w}^\top \mathbf{x} + b$. Using Lemma 2.9.1, we have

$$\mathbf{w}^\top \mathbf{x} + b = \left(\sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)} \right)^\top \mathbf{x} + b.$$

Therefore,

$$\mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^n \alpha_i y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{x} + b.$$

The prediction is

$$\hat{y} = \text{sign} \left(\sum_{i=1}^n \alpha_i y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{x} + b \right).$$

Since most $\alpha_i = 0$, the prediction depends only on support vectors.

2.13 Kernelised SVM

The hard-margin dual depends on the data only through inner products $(\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)}$. Therefore, we can replace inner products by kernels $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$.

Kernelised dual

$$\begin{aligned} \max_{\boldsymbol{\alpha}} W(\boldsymbol{\alpha}) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y^{(i)} y^{(j)} \alpha_i \alpha_j K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \\ \text{s.t. } \alpha_i &\geq 0, \quad i = 1, \dots, n, \\ \sum_{i=1}^n \alpha_i y^{(i)} &= 0. \end{aligned}$$

Kernelised prediction

$$\hat{y} = \text{sign} \left(\sum_{i=1}^n \alpha_i y^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}) + b \right).$$

This allows SVMs to operate in high-dimensional or infinite-dimensional feature spaces without explicitly computing the feature map.

2.14 Regularisation and the non-separable case

The hard-margin SVM assumes the data is linearly separable. In practice, data may not be separable, or there may be outliers.

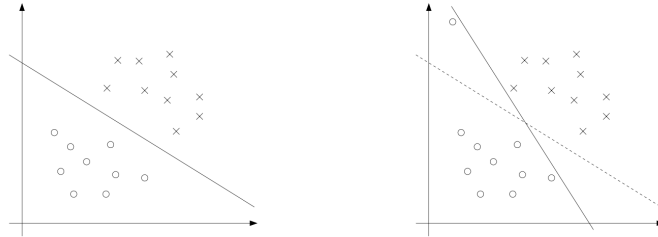


Figure 3: Adding a single outlier dramatically changes the optimal separating hyperplane. This motivates soft-margin SVM.

Definition 2.14.1 (Slack variables). Introduce slack variables $\xi_i \geq 0, i = 1, \dots, n$. The constraint becomes

$$y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i.$$

- If $\xi_i = 0$, the example satisfies the margin constraint.
- If $0 < \xi_i < 1$, the example is correctly classified but inside the margin.

- If $\xi_i \geq 1$, the example may be misclassified.

Theorem 2.14.1 (Soft-margin SVM primal). *The soft-margin SVM solves*

$$\begin{aligned} \min_{\xi, \mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \dots, n, \\ & \xi_i \geq 0, \quad i = 1, \dots, n. \end{aligned}$$

The parameter $C > 0$ controls the trade-off between:

1. maximising the margin by keeping $\|\mathbf{w}\|^2$ small;
2. penalising margin violations through $\sum_i \xi_i$.

Large C penalises violations strongly. Small C allows more violations but may produce a wider margin.

Soft-margin Lagrangian

The Lagrangian is

$$\mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mathbf{r}) = \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i \left[y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) - 1 + \xi_i \right] - \sum_{i=1}^n r_i \xi_i.$$

Here, $\alpha_i \geq 0$, $r_i \geq 0$.

Theorem 2.14.2 (Soft-margin SVM dual). *The soft-margin SVM dual is*

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y^{(i)} y^{(j)} \alpha_i \alpha_j (\mathbf{x}^{(i)})^\top \mathbf{x}^{(j)} \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n \alpha_i y^{(i)} = 0. \end{aligned}$$

Compared with the hard-margin dual, the only change is the upper bound $a_i \leq C$.

Kernelised soft-margin dual

With kernels, the dual becomes

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y^{(i)} y^{(j)} \alpha_i \alpha_j K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n \alpha_i y^{(i)} = 0. \end{aligned}$$

2.15 KKT conditions for soft-margin SVM

For the soft-margin SVM, the KKT conditions imply:

- $\alpha_i = 0 \implies y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1.$
- $\alpha_i = C \implies y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \leq 1.$
- $0 < \alpha_i < C \implies y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 1.$

These conditions are important for understanding support vectors and for checking convergence in SMO.

Interpretation

- If $\alpha_i = 0$, the point is not a support vector.
- If $0 < \alpha_i < C$, the point lies exactly on the margin.
- If $\alpha_i = C$, the point is inside the margin or misclassified.

2.16 Coordinate ascent

Suppose we want to solve

$$\max_{\alpha} W(\alpha_1, \dots, \alpha_n).$$

Coordinate ascent repeatedly optimises one coordinate at a time while holding all others fixed.

Algorithm 1 Coordinate Ascent

Repeat until convergence:

for $i = 1, \dots, n$ **do**

$\alpha_i \leftarrow \underset{\hat{\alpha}_i}{\operatorname{argmax}} W(\alpha_1, \dots, \alpha_{i-1}, \hat{\alpha}_i, \alpha_{i+1}, \dots, \alpha_n)$

end for

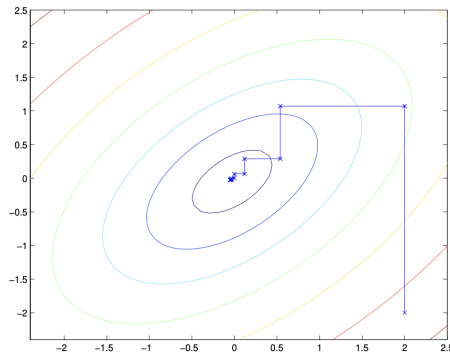


Figure 4: Coordinate ascent on a quadratic objective. The path moves parallel to coordinate axes because only one variable is updated at a time.

2.17 SMO algorithm

To solve the SVM dual problem, the equality constraint means we cannot update just one α_i at a time.

Lemma 2.17.1. *Suppose $\alpha_2, \dots, \alpha_n$ are fixed. Then α_1 is determined by the equality constraint*

$$\sum_{i=1}^n \alpha_i y^{(i)} = 0.$$

Proof. The equality constraint gives

$$\alpha_1 y^{(1)} + \sum_{i=2}^n \alpha_i y^{(i)} = 0.$$

Therefore,

$$\alpha_1 y^{(1)} = - \sum_{i=2}^n \alpha_i y^{(i)}.$$

Since $y^{(1)} \in \{-1, 1\}$, we have $(y^{(1)})^2 = 1$. Multiplying both sides by $y^{(1)}$,

$$\alpha_1 = -y^{(1)} \sum_{i=2}^n \alpha_i y^{(i)}.$$

Thus, if all other α_i 's are fixed, α_1 is fixed. Therefore, updating only one coordinate would violate the equality constraint.

SMO main idea

Because one coordinate cannot be updated alone, SMO updates two coordinates at a time.

Algorithm 2 SMO Algorithm

Repeat until convergence:

1. Select a pair α_i, α_j .
 2. Optimise $W(\alpha)$ with respect to α_i, α_j , holding all other α_k fixed.
-

Convergence is tested using the KKT conditions, usually up to a tolerance such as 0.01 or 0.001.

2.18 Two-variable SMO update

Suppose we update α_1 and α_2 while holding $\alpha_3, \dots, \alpha_n$ fixed. The equality constraint becomes

$$\alpha_1 y^{(1)} + \alpha_2 y^{(2)} = - \sum_{i=3}^n \alpha_i y^{(i)}.$$

Let

$$\zeta = - \sum_{i=3}^n \alpha_i y^{(i)}.$$

Then

$$\alpha_1 y^{(1)} + \alpha_2 y^{(2)} = \zeta. \quad (6)$$

This means (α_1, α_2) must lie on a line.

At the same time, box constraints require $0 \leq \alpha_1 \leq C$ and $0 \leq \alpha_2 \leq C$.

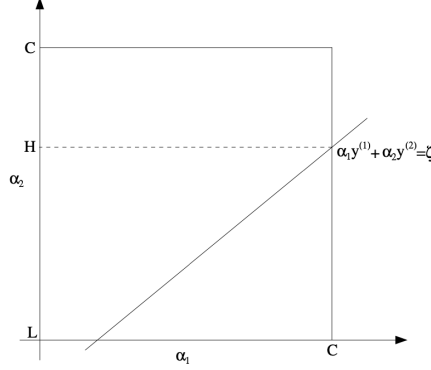


Figure 5: The feasible region of α_1 and α_2 : a box $[0, C] \times [0, C]$ intersected with the line $\alpha_1 y^{(1)} + \alpha_2 y^{(2)} = \zeta$.

Thus, α_2 must lie in an interval $L \leq \alpha_2 \leq H$.

Expressing α_1 in terms of α_2

From equation 6, we get

$$\alpha_1 y^{(1)} = \zeta - \alpha_2 y^{(2)}.$$

Multiplying by $y^{(1)}$,

$$\alpha_1 = (\zeta - \alpha_2 y^{(2)}) y^{(1)}.$$

Thus, the objective becomes a function of only α_2 :

$$W(\alpha_1, \alpha_2, \dots, \alpha_n) = W((\zeta - \alpha_2 y^{(2)}) y^{(1)}, \alpha_2, \dots, \alpha_n).$$

Since all other variables are fixed, this is a quadratic function of α_2 .

Clipping step

Ignoring box constraint, the best value is obtained by solving the one-dimensional quadratic optimisation. Let this be $\alpha_2^{\text{new,unclipped}}$. Then we clip it into the feasible interval:

$$\alpha_2^{\text{new}} = \begin{cases} H & \text{if } \alpha_2^{\text{new,unclipped}} > H, \\ \alpha_2^{\text{new,unclipped}} & \text{if } L \leq \alpha_2^{\text{new,unclipped}} \leq H, \\ L & \text{if } \alpha_2^{\text{new,unclipped}} < L. \end{cases}$$

After finding α_2^{new} , compute α_1^{new} using

$$\alpha_1 = (\zeta - \alpha_2 y^{(2)}) y^{(1)}.$$

3 References and Further Reading

These notes follow the treatment of kernel methods and support vector machines in the CS229 lecture notes by Andrew Ng and Tengyu Ma [1].

References

- [1] A. Ng and T. Ma, “CS229 Lecture Notes: Kernel Methods and Support Vector Machines.” Stanford University, 2023. Chapters 5–6.